

Day 4 — Structured Logic: Full Instructor Teaching Material (Technical Edition)

Total session time: 210 minutes (3.5 hrs). Each section is sized to its time budget with talking points, board work, worked ST/ladder code, register-level detail, and explicit framing lines.

1. Shift Registers — 30 min

1a. Concept — 10 min

Precise definition: a shift register tracks an ordered sequence of items moving through discrete stations by moving bit or word values one position through an array each scan or each trigger event — modeling something physically moving down a line without needing a dedicated sensor-tracking scheme for every single station.

Two implementation forms worth naming explicitly, since students will meet both on real platforms:

- **Dedicated bit-shift instruction** — many platforms (Allen-Bradley's BSL/BSR, "Bit Shift Left"/"Bit Shift Right") shift every bit in a single word/array one position on each rung-true transition, inserting a new source bit at one end.
- **Manual array-shift in Structured Text** — build it explicitly with an array tag and ordered assignment statements, which is the form worth teaching first since it makes the underlying mechanism visible rather than hiding it inside a single opaque instruction.

Manual 4-station shift register, ST form:

```
// Word-array based shift register - 4 stations, index 0 = newest
Station[3] := Station[2];
Station[2] := Station[1];
Station[1] := Station[0];
Station[0] :=NewItemPresent; // BOOL - true the scan a new item enters station 1
```

Ordering dependency, stated as a hard rule: the shift must happen from the far end toward the near end — `Station[3]` must be written before `Station[2]`, which must be written before `Station[1]`, before finally overwriting `Station[0]` with the new value. Writing in the opposite order would overwrite `Station[0]` first and then copy that same new value into every subsequent slot on the same scan, destroying the entire sequence instead of shifting it — this is the exact same ordering hazard flagged in Day 3's moving-average filter (Block 5, "if finished early"), now generalized to a full shift register rather than a fixed 3-sample average.

Dedicated bit-shift instruction form (Allen-Bradley style, for contrast):

```
BSL(Array := StationTrack, Control := ShiftControl, SourceBit :=NewItemPresent,
Length := 4);
```

State explicitly: the instruction performs exactly the same far-to-near shift internally — walking through the manual ST version first is what prevents BSL from being an unexplainable black box, matching the same pedagogical reasoning used for `SCL` / `SCALE_X` on Day 3.

1b. Concrete Framing — Conveyor With 4 Stations — 8 min

Imagine 4 stations on a conveyor, `Station[0]` nearest the entry sensor and `Station[3]` nearest the exit. Walk through three scans on the board:

```
Scan 0: Station = [0,0,0,0] (line empty)
Scan 1: item A enters -> [1,0,0,0] (Station[0] set true)
Scan 2: item B enters -> [1,1,0,0] (A shifts to Station[1], B enters Station[0])
Scan 3: item C enters -> [1,1,1,0] (A shifts to Station[2], B to Station[1], C enters)
```

Note explicitly: this is not a real-time shift tied to conveyor speed — it is triggered once per discrete event (a photoeye detecting a new item), not once per scan. If the register were shifted every scan regardless of whether a new item actually arrived, every item would appear to move station-to-station on every single scan, which does not correspond to anything physical. The trigger condition (photoeye rising edge) must gate the shift, directly reusing the `R_TRIG` /edge-detection pattern from Day 2, Block 2c.

1c. Live Demo — 6 min

Simulate a 4-station line with a pushbutton standing in for the entry photoeye, wired through an `ONS` / `R_TRIG` into the shift rung above. Press the button several times and watch the array values move down the row in the software's live data view. Ask students to predict, before each press, what the array will look like after the shift — turning the demo into a prediction-then-verify exercise, matching the pattern established for edge-detection demos on Day 2.

1d. Project: Simulate 4 Stations, Push a Bottle Through — 6 min

Task: wire a simulated entry sensor through edge detection into the 4-element shift register above, and display all four station values on screen. While circulating, check specifically: is the shift wired in the correct far-to-near order? Ask a student to predict the array state after exactly 5 presses (should read all four stations occupied, since the register only holds 4 positions — a natural lead-in to asking what happens on the 5th press: the oldest tracked item simply falls off the far end, since `Station[3]` 's prior value is overwritten and never copied anywhere).

2. Program Control — Subroutines & Function Blocks — 40 min

2a. Why Structured Code — 10 min

Everything built Days 1–3 has been flat rungs in one continuous program — workable for small projects, but it does not scale. State the problem concretely: the Day 3 temperature-monitoring pipeline (scale → compare → alarm) written flat would need to be copy-pasted, with every tag renamed, for a second independent sensor. A **Function Block (FB)** solves this: a reusable, self-contained piece of logic with its own declared inputs and outputs that can be called multiple times against different data, where each call gets its own dedicated block of memory (its **instance data**) holding that call's internal state independently of every other call.

Contrast explicitly with a **Function (FC)**: stateless, purely input-to-output, with no memory retained between calls — calling it twice with the same inputs always produces the same outputs, unlike an FB, which can hold a timer's accumulated value or similar persistent state across scans.

Aspect	Function Block (FB)	Function (FC)
Retains memory between calls?	Yes — own instance data block per call	No — stateless
Can contain a TON/CTU/etc. internally?	Yes	Generally no (no persistent state to hold ET/CV)
Typical use	Anything needing to remember something: a timer, a counter, a running state	Pure calculation: unit conversion, a formula with no memory need
Call syntax	Named instance required, e.g. `Sensor1(...)`	Called directly as an expression, e.g. `CtoF(50.0)`

When to use which — worth stating explicitly, since it is a genuinely common interview question: use an FB whenever the logic needs to remember something between scans; use an FC for a pure, memory-free calculation.

2b. FB Syntax & Instance Data — 10 min

Worked example — package Day 3's temperature-monitoring pipeline as an FB:

```
FUNCTION_BLOCK TempMonitorFB
VAR_INPUT
    RawADC : INT;
    Setpoint : REAL;
END_VAR
VAR_OUTPUT
    ScaledTemp : REAL;
    HighTempAlarm : BOOL;
END_VAR
VAR
    LogTrigger : BOOL; // internal, not exposed outside the FB
END_VAR

SCL(Source := RawADC, InputMin := 0, InputMax := 1023,
    ScaledMin := 0.0, ScaledMax := 100.0, Dest := ScaledTemp);
HighTempAlarm := ScaledTemp > Setpoint;
END_FUNCTION_BLOCK
```

Calling it twice, for two independent sensors:

```
Sensor1(RawADC := RawADC_1, Setpoint := 50.0);
Sensor2(RawADC := RawADC_2, Setpoint := 60.0);

// downstream logic reads each instance's own output data independently:
HighAlarmZone1 := Sensor1.HighTempAlarm;
HighAlarmZone2 := Sensor2.HighTempAlarm;
```

State explicitly why this matters, not just how it works: `Sensor1` and `Sensor2` are two separate named instances of the same `TempMonitorFB` definition, each with its own private copy of every `VAR` / `VAR_OUTPUT` tag declared inside the FB. Changing `Sensor1.ScaledTemp` on a given scan has zero effect on `Sensor2.ScaledTemp` — this is exactly what makes the same block of logic reusable across many sensors without them interfering with each other's internal state, the same isolation principle that lets two independent `TON` instances run on completely different schedules without one resetting the other.

2c. FC (Function) Contrast — 8 min

Stateless calculation example — the °C→°F conversion from Day 3, packaged as an FC rather than an FB, since it holds no memory:

```
FUNCTION CtoF : REAL
VAR_INPUT
    Celsius : REAL;
END_VAR
CtoF := Celsius * 1.8 + 32.0;
END_FUNCTION
```

Called directly as an expression, with no named instance required:

```
Temp1_F := CtoF(Sensor1.ScaledTemp);
Temp2_F := CtoF(Sensor2.ScaledTemp);
```

State explicitly: there is no `CtoF1` / `CtoF2` distinction the way there was for `Sensor1` / `Sensor2` — because an FC has no internal state to keep separate between calls, a single FC definition can be called from anywhere in the program with no instance bookkeeping at all. Ask the class directly: "could `TempMonitorFB` have been written as an FC instead?" (No — it needs to retain `ScaledTemp` and `HighTempAlarm` as durable outputs other logic reads across scans, and in a fuller version would hold a `TON` / `R_TRIG` for alarm logging exactly as Day 3's project did, which requires persistent state an FC cannot hold.)

2d. Program Organization — POU Hierarchy — 6 min

Introduce the term **POU** (Program Organization Unit) — the umbrella term covering programs, function blocks, and functions as callable units. Structure to put on the board:

```
Main Program (OB1 / top-level)
|-- calls --> Sensor1 : TempMonitorFB (instance 1)
|-- calls --> Sensor2 : TempMonitorFB (instance 2)
|-- calls --> CtoF(...) (stateless, no instance)
```

State the organizing principle explicitly, since it is a professional norm worth establishing now: keep the Main/top-level program thin — mostly a sequence of FB calls and high-level wiring between their outputs — and push the actual logic detail down into FBs and FCs. A Main program that is one giant wall of flat rungs (the style used through Day 3, appropriate for learning fundamentals) becomes unmaintainable at real project scale; today's shift toward FBs is the direct professional-practice fix for that.

2e. Project: Convert Day 3's Monitor Into an FB, Call It Twice — 6 min

Task: implement `TempMonitorFB` exactly as sketched in 2b, instantiate it twice as `Sensor1` / `Sensor2` with independent simulated raw inputs and independent setpoints, and confirm — by forcing each sensor's raw input independently using Block 3's forcing tools from Day 2 — that each instance's `HighTempAlarm` output responds only to its own sensor, never the other's.

3. PID Basics — 30 min

3a. Concept Only — Full Tuning Is Out Of Scope This Week, Be Upfront About That — 15 min

PID stands for Proportional–Integral–Derivative — a control algorithm that continuously computes a correction output to drive a measured **process variable (PV)** toward a desired **setpoint (SP)**, based on the error between them: $e(t) = SP - PV$.

Continuous-form equation, worth putting on the board even though the PLC implementation is discrete:

$$\text{Output}(t) = K_p * e(t) + K_i * \text{Integral}[e(t) \, dt] + K_d * d(e(t))/dt$$

The three terms, defined precisely:

- **P (Proportional)** — output responds directly to the current error. A bigger error produces a bigger immediate correction. `Kp` is the proportional gain — a tuning constant scaling how aggressively the controller reacts.
- **I (Integral)** — accumulates error over time and adds a correction proportional to that accumulated total, which is what eliminates a small persistent steady-state offset a P-only controller cannot fully remove (e.g., a heater that settles slightly below setpoint indefinitely under P-only control because the small remaining error alone never produces enough proportional output to close the gap).
- **D (Derivative)** — responds to the *rate of change* of the error, adding a correction that opposes fast-approaching error, which dampens overshoot by anticipating where the process variable is heading rather than reacting only to where it currently is.

Discrete approximation actually used inside a PLC scan, worth showing since students have only seen continuous calculus notation so far:

```
e[n] := SP - PV;  
Integral := Integral + e[n] * dt;  
Derivative := (e[n] - e_prev) / dt;  
Output[n] := (Kp * e[n]) + (Ki * Integral) + (Kd * Derivative);  
e_prev := e[n];
```

Analogy to use, extending the one already introduced Day 2/3 for timers: driving a car to hold a speed limit. P is "how hard you press the pedal based on how far off you currently are." I is "you've been slightly under the limit for a while now, so nudge the pedal up a bit more to compensate for that lingering gap." D is "you're approaching the limit quickly, so ease off preemptively before you overshoot it."

Integral windup — a genuine, concrete bug worth naming explicitly, not just a footnote: if the process variable cannot reach setpoint for an extended period (e.g., a heater at maximum output still below setpoint on a cold start), the `Integral` term keeps accumulating without bound the entire time, producing a very large `Ki *`

Integral term. Once the process finally does approach setpoint, this oversized accumulated term keeps driving the output high well past where it should — a significant, visible overshoot caused entirely by the integral term's own bookkeeping, not by the process itself. The standard fix is **anti-windup clamping** — capping the **Integral** accumulator at a maximum magnitude, or freezing accumulation once output saturates — worth naming as the standard mitigation even without implementing it this week.

3b. Built-In PID Instruction — 8 min

Most platforms provide a built-in PID function block rather than requiring the raw math above to be hand-written every time — shown here specifically so it is not treated as a black box, matching the same reasoning already used for `SCL / SCALE_X` and `BSL` :

```
PID_Instance(SP := Setpoint, PV := ScaledTemp,
             Kp := 2.0, Ki := 0.5, Kd := 0.1,
             MANUAL := FALSE, OUT => HeaterOutput);
```

Note the **MANUAL** input — most industrial PID blocks support a manual/auto mode, where in manual mode **OUT** is driven directly by an operator-entered value instead of the algorithm, and the block internally tracks the transition (a "bumpless transfer" concern — worth a one-line mention, full depth out of scope). **OUT** is typically a 0–100% value mapped onto an analog output (0–10V or 4–20mA, directly reusing Day 3's analog-output scaling in reverse — engineering value out rather than in).

3c. Simulated Demo — 7 min

Show a PID loop responding live to a step change in setpoint (e.g., a simulated heater jumping from a 40°C to 70°C setpoint). Point out the characteristic response curve on the board: an initial fast rise (P dominating), a possible small overshoot past 70°C before settling (D restraining most of it, I closing the final small gap), and a final settled value sitting exactly at setpoint. State plainly: no hands-on tuning is required this week — the goal is recognizing the shape of a PID response and naming which term is responsible for which visible feature, not producing a tuned loop. Mention Ziegler–Nichols by name only, as a "where to go next" pointer for students who want to pursue tuning further, matching the same forward-pointer treatment OPC UA and Structured Text get in Friday's wrap-up.

4. Day Project — Batch Mixing Station — 110 min

Goal, kept on the board: a 3-stage sequential process — Fill (timed) → Mix (motor + timed) → Drain (level-sensor-based) — built as separate function blocks called from one main sequencer, tying together today's FB content with the sequencing/state-variable concept from Day 2's traffic light project.

1. Design — 10 min

State diagram to sketch:

```
[Idle] --StartBtn--> [Fill, TON PT=8s] --Q--> [Mix, Motor+TON PT=15s] --Q--> [Drain,
LevelEmpty] --> back to [Idle]
```

Have students annotate each state with the concrete FB instance and output tag active during it — exactly the same "map abstract state to concrete tags before writing code" discipline established for the Day 2 traffic-light project.

State explicitly, as a framing point before design begins: the Drain stage is the first state transition all week driven by something other than a timer — a level sensor reaching a defined empty condition — worth flagging now so students do not assume every state machine is timer-gated by default.

2. Build Fill FB — 25 min

```
FUNCTION_BLOCK FillStationFB
VAR_INPUT
    StartFill : BOOL;
END_VAR
VAR_OUTPUT
    FillValve : BOOL;
    FillDone : BOOL;
END_VAR
VAR
    FillTimer : TON;
END_VAR

FillTimer(IN := StartFill, PT := T#8s);
FillValve := StartFill AND NOT FillTimer.Q;
FillDone := FillTimer.Q;
END_FUNCTION_BLOCK
```

Note explicitly: `FillValve` is gated `NOT FillTimer.Q` — the valve is meant to close the instant the fill timer completes, not stay open indefinitely. Ask the class to predict what would happen if that `NOT FillTimer.Q` term were omitted (the valve would remain open forever once `StartFill` first goes true, since nothing in this rung as written would ever close it — a genuinely common first-draft mistake worth flagging proactively while circulating).

3. Build Mix FB — 25 min

```
FUNCTION_BLOCK MixStationFB
VAR_INPUT
    StartMix : BOOL;
END_VAR
VAR_OUTPUT
    MixMotor : BOOL;
    MixDone : BOOL;
END_VAR
VAR
    MixTimer : TON;
END_VAR

MixTimer(IN := StartMix, PT := T#15s);
MixMotor := StartMix AND NOT MixTimer.Q;
MixDone := MixTimer.Q;
END_FUNCTION_BLOCK
```

State explicitly: this FB is structurally identical to `FillStationFB` — same pattern, different preset and different output tag names — worth having students notice the repetition themselves rather than pointing it out first, since recognizing "this is the same pattern with different tags" is exactly the instinct that makes someone reach

for an FB in the first place instead of writing bespoke logic for every stage.

4. Build Drain FB — 25 min

```
FUNCTION_BLOCK DrainStationFB
VAR_INPUT
    StartDrain : BOOL;
    LevelEmpty : BOOL; // TRUE once the tank level sensor confirms empty
END_VAR
VAR_OUTPUT
    DrainValve : BOOL;
    DrainDone : BOOL;
END_VAR

DrainValve := StartDrain AND NOT LevelEmpty;
DrainDone := StartDrain AND LevelEmpty;
END_FUNCTION_BLOCK
```

Call out explicitly, since it is the structural novelty of this block: there is no `TON` anywhere in this FB — the transition condition is a physical sensor reading, not elapsed time. This is deliberately built right after two timer-based FBs so the contrast is visible immediately: a state transition can be gated by any `BOOL` condition the process actually provides, timers are simply the most common one, not the only one.

5. Integrate & Test — 25 min

Main sequencer, using an `INT` state register directly reusing Day 2's traffic-light `TrafficState` / `EQU` pattern:

```
IF BatchState = 0 AND StartBtn THEN BatchState := 1; END_IF; // Idle -> Fill

Filler(StartFill := BatchState = 1, ...);
IF BatchState = 1 AND Filler.FillDone THEN BatchState := 2; END_IF; // Fill -> Mix

Mixer(StartMix := BatchState = 2, ...);
IF BatchState = 2 AND Mixer.MixDone THEN BatchState := 3; END_IF; // Mix -> Drain

Drainer(StartDrain := BatchState = 3, LevelEmpty := LevelEmptySensor);
IF BatchState = 3 AND Drainer.DrainDone THEN BatchState := 0; END_IF; // Drain -> Idle
```

Call all three FBs from this one main sequencer and verify the full `Fill` → `Mix` → `Drain` cycle runs correctly end to end, using Block 3's online monitoring habits from Day 2 to watch `BatchState` and each FB's `.Done` output transition in real time.

Common bug to watch for, stated explicitly: forgetting to reset a completed stage's internal timer/state before the next cycle begins — e.g., if `Filler.FillTimer` is not naturally re-armed by `StartFill` going false and true again, the second run of the batch can behave differently from the first. Since each FB's `TON` already resets `ET` to zero whenever its own `IN` (here, `StartFill`) goes false — a direct callback to Day 2 Block 1a's "ET resets the scan after IN goes false" rule — this generally self-corrects as long as `StartFill` genuinely returns to false between cycles; have students verify this explicitly on the `.ET` monitor rather than assuming it, the same numeric-value-first troubleshooting habit established Day 2, Block 4b.

If finished early: add a manual/auto mode switch — in auto, the FB sequence above runs unattended; in manual, an operator can directly jog the mixer motor output regardless of `BatchState` . This introduces mode-selection/interlock logic (auto sequencing and manual override must never both be able to drive `MixMotor` at once) — a pattern used constantly in real industrial programs, and a direct forward-seed for the safety-interlocking content on Day 5.